
Probeklausur 1

Grundlagen der Programmierung

Studiengänge: MIB & OMB

Autor: Luis Staudt

Semester: Sommersemester 2026

Bearbeitungszeit: 90 Minuten

Gesamtpunkte: 90

Aufgabe	1	2	3	4	Gesamt
Punkte	25	20	20	25	90
Erreichte Punkte					

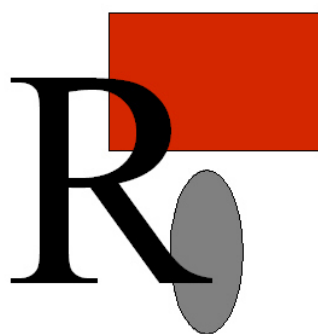
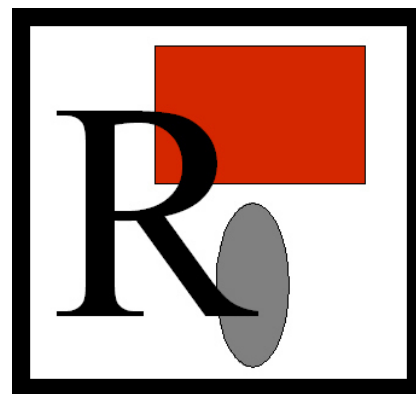
Aufgabe 1: Bildverarbeitung – Rahmen schwarz färben**(25 Punkte)**

Ein Bitmap werde in einem zweidimensionalen `int`-Array `p` gespeichert. Jede Farbe wird durch eine Zahl vom Typ `int` dargestellt. Die Variable `p[x][y]` steht für die Farbe des Bildes an der Koordinate (x, y) . Der Ursprung des Koordinatensystems liegt links oben, somit enthält `p[0][0]` die Farbe des Pixels in der linken oberen Ecke.

<code>p[0][0]</code>	<code>p[1][0]</code>	<code>p[2][0]</code>	<code>p[3][0]</code>	<code>p[4][0]</code>	<code>p[5][0]</code>
<code>p[0][1]</code>	<code>p[1][1]</code>	<code>p[2][1]</code>	<code>p[3][1]</code>	<code>p[4][1]</code>	<code>p[5][1]</code>
<code>p[0][2]</code>	<code>p[1][2]</code>	<code>p[2][2]</code>	<code>p[3][2]</code>	<code>p[4][2]</code>	<code>p[5][2]</code>
<code>p[0][3]</code>	<code>p[1][3]</code>	<code>p[2][3]</code>	<code>p[3][3]</code>	<code>p[4][3]</code>	<code>p[5][3]</code>
<code>p[0][4]</code>	<code>p[1][4]</code>	<code>p[2][4]</code>	<code>p[3][4]</code>	<code>p[4][4]</code>	<code>p[5][4]</code>

Die Breite des Bildes ist `p.length`, die Höhe ist `p[0].length`. Damit liegt x im Bereich 0 bis `p.length-1` und y im Bereich 0 bis `p[0].length-1`. Der Wert 0 steht für die Farbe schwarz.

Schreiben Sie eine Klassenmethode `f`, die einen **Rahmen der Dicke 15 Pixel** schwarz färbt: Alle Pixel, deren Abstand zum Bildrand weniger als 15 Pixel beträgt (d. h. die in den äußersten 15 Spalten bzw. Zeilen liegen), sollen auf 0 gesetzt werden. Die Methode `f` soll als Parameter `p` einen zweidimensionalen `int`-Array haben und keinen Rückgabewert.

*Originalbild**Zielbild nach Aufruf von `f(p)`*

Aufgabe 2: Syntaxfehler – Klasse Konto**(20 Punkte)**

Der Programmcode der Klasse `Konto` enthält mehrere syntaktische Fehler. Geben Sie zu jedem Fehler die Zeilennummer der Zeile an, in der sich der Fehler befindet, und beschreiben Sie jeweils den Fehler.

```
1 public class Konto {
2
3     public double stand;
4     private String inhaber
5
6     public Konto(String name) {
7         inhaber = name;
8         stand = 0.0;
9     }
10
11     public void einzahlen(double betrag) {
12         if (betrag > 0)
13             stand = stand + betrag;
14         return stand;
15     }
16
17     public boolean abheben(double betrag) {
18         if (betrag <= stand) {
19             stand = stand - betrag;
20             return true;
21         }
22     }
23
24     public static void main(String[] args) {
25         Konto k = new Konto("Anna");
26         k.einzahlen(100);
27         double rest = k.abheben(30);
28         System.out.println(k.stand);
29         if (k.stand > 0)
30             System.out.println("Guthaben: " k.stand);
31     }
32 }
```

Zeile	Fehler

Aufgabe 3: Programmausgabe – rekursive Methode**(20 Punkte)**

Zu welcher Ausgabe kommt es, wenn man die Methode `main` der Klasse `X` ausführt?

```
1 public class X {  
2  
3     public static void f(int n) {  
4         if (n <= 0)  
5             return;  
6         System.out.print(n + " ");  
7         f(n - 1);  
8         System.out.print(n + " ");  
9     }  
10  
11     public static void main(String[] args) {  
12         for (int i = 1; i <= 3; i++) {  
13             f(i);  
14             System.out.println();  
15         }  
16     }  
17 }
```

Hinweis. `System.out.print` erzeugt eine Ausgabe ohne anschließenden Zeilenumbruch, `System.out.println()` erzeugt einen Zeilenumbruch.

Empfehlung. Bestimmen Sie zunächst getrennt die Ausgabe der Aufrufe `f(1)`, `f(2)` und `f(3)` und setzen Sie diese anschließend zur Gesamtausgabe zusammen.

Aufgabe 4: Methode implementieren – Nachbarsummen**(25 Punkte)**

Schreiben Sie eine statische Methode `f`, die einen Parameter `a` vom Typ `int`-Array hat und keinen Rückgabewert.

Die Methode `f` soll jeden Wert des Arrays durch die **Summe seiner beiden direkten Nachbarn** ersetzen. Das erste und das letzte Element des Arrays haben nur einen Nachbarn und sollen **unverändert** bleiben.

Hinweis. Überlegen Sie, weshalb Sie zur korrekten Berechnung eine Kopie der ursprünglichen Werte benötigen.

a vor der Ausführung von <code>f(a)</code>	a nach der Ausführung von <code>f(a)</code>
[3, 1, 4, 1, 5]	[3, 7, 2, 9, 5]
[2, 2, 2]	[2, 4, 2]
[7, 8]	[7, 8]
[10]	[10]